

AWS VPC TASK 2

1. Architecture Overview

This guide covers building one base VPC with one public and one private subnet, then connecting that VPC to other networks using three different AWS connectivity patterns: cross-Region VPC peering, cross-account VPC peering, a Transit Gateway hub, and finally a VPC Endpoint for private access to an AWS service.

1.1 Components Built in This Guide

#	Component	Details
1	Base VPC	1 public subnet + 1 private subnet, single region
2	Cross-Region Peering	Base VPC (Region A) peered with a second VPC in Region B
3	Cross-Account Peering	Base VPC peered with a VPC in a friend's/second AWS account
4	Transit Gateway	Central hub attaching multiple VPCs instead of point-to-point peering
5	VPC Endpoint	Private connectivity to an AWS service (e.g., S3) without using the internet

- tables.

2. Create the Base VPC (1 Public + 1 Private Subnet)

STEP 1 Create the VPC

- VPC Console → Your VPCs → Create VPC → Resources to create: VPC only.
- Name tag: my-vpc
- IPv4 CIDR block: 192.168.0.0/24
- Click Create VPC.

SCREENSHOT: Base VPC created

vpc-074b41b4d7bee7853 / My-vpc

Actions

Details

VPC ID

vpc-074b41b4d7bee7853

DNS resolution

Enabled

Main network ACL

acl-0a27b7a685facf8de

IPv6 CIDR (Network border group)

-

Encryption control ID

-

State

Available

Tenancy

default

Default VPC

No

Network Address Usage metrics

Disabled

Encryption control mode

-

Block Public Access

Off

DHCP option set

dopt-0c71fc5639e429bd1

IPv4 CIDR

192.168.0.0/24

Route 53 Resolver DNS Firewall rule groups

-

DNS hostnames

Disabled

Main route table

rtb-053e0026f3f43ac8f

IPv6 pool

-

Owner ID

565398309825

Resource map

CIDRs

Flow logs

Tags

Related resources

Integrations

Resource map

Show all details

STEP 2 Enable DNS Hostnames

- Select my-vpc → Actions → Edit VPC settings.
- Check “Enable DNS hostnames” → Save changes.

STEP 3 Create the Public and Private Subnets

Subnet name	Availability Zone	CIDR block
base-public-subnet	AZ-a	192.168.0.0/25
base-private-subnet	AZ-a	192.168.0.0/25

- VPC Console → Subnets → Create subnet → select my-vpc → add both rows above → Create subnet.

SCREENSHOT: Both subnets created

The screenshot shows the AWS VPC Subnets console. At the top, it says "Subnets (2)" with an "Info" link. There's a search bar "Find subnets by attribute or tag". On the right, it says "Last updated less than a minute ago" with a refresh icon, and buttons for "Actions" and "Create subnet". Below this is a table with columns: Name, Subnet ID, State, VPC, and Block Public... The table contains two rows: "pub-subnet1" with Subnet ID "subnet-0ffdcf38d45339456", State "Available", VPC "vpc-074b41b4d7bee7853", and Block Public... "Off"; and "priv-subnet1" with Subnet ID "subnet-08b8a90d19ab30750", State "Available", VPC "vpc-074b41b4d7bee7853", and Block Public... "Off".

STEP 4 Enable Auto-Assign Public IP on the Public Subnet

- Select base-public-subnet → Actions → Edit subnet settings.
- Check “Enable auto-assign public IPv4 address” → Save.

STEP 5 Create and Attach an Internet Gateway

- VPC Console → Internet gateways → Create internet gateway → name: base-igw.
- Select base-igw → Actions → Attach to VPC → base-vpc.

SCREENSHOT: Internet gateway attached

The screenshot shows the AWS VPC Internet Gateway console. At the top, it says "igw-07d4a51e63615e5da / My-IGW" with an "Actions" button. Below this is a "Details" section with "Info" link. It shows "Internet gateway ID" as "igw-07d4a51e63615e5da", "State" as "Attached", "VPC ID" as "vpc-01de7bd764413ad71 | ohio-vpc", and "Owner" as "565398309825". Below this is a "Tags (1)" section with a "Manage tags" button. It shows a search bar "Search tags" and a table with columns "Key" and "Value". The table contains one row: "Name" with value "My-IGW".

STEP 6 Create Route Tables

- Create base-public-rt → add route 0.0.0.0/0 → Target: base-igw → associate with base-public-subnet.

- Create base-private-rt → leave only the local route for now → associate with base-private-subnet.

SCREENSHOT: Route tables configured

igw-07d4a51e63615e5da / My-IGW

Actions ▾

Details [Info](#)

Internet gateway ID
igw-07d4a51e63615e5da

State
Attached

VPC ID
[vpc-01de7bd764413ad71 | ohio-vpc](#)

Owner
565398309825

Tags (1)

Manage tags

Search tags

Key	Value
Name	My-IGW

NOTE: The private-subnet route table will be extended later in this guide with routes to the Transit Gateway and/or peering connections.

3. Enable VPC Peering — Cross-Region

This section peers base-vpc (Region A) with a second VPC (VPC-B) created in a different Region.

STEP 7 Create the Second VPC in Region B

- Switch the console Region selector to Region B (e.g., us-east-1).
- Create a VPC named peer-vpc-region-b with CIDR 10.1.0.0/16 (with at least one subnet, e.g., 10.1.1.0/24).

SCREENSHOT: Region B VPC created

Instance summary for i-01ef08a7335ea7bf9 (virginia-vpc) [Info](#)

Updated less than a minute ago

Instance ID
i-01ef08a7335ea7bf9

IPv6 address
-

Hostname type
IP name: ip-192-168-0-23.ec2.internal

Answer private resource DNS name
-

Auto-assigned IP address
44.202.184.29 [Public IP]

Public IPv4 address
44.202.184.29 | [open address](#)

Instance state
Running

Private IP DNS name (IPv4 only)
ip-192-168-0-23.ec2.internal

Instance type
t3.micro

VPC ID
vpc-074b41b4d7bee7853 (My-vpc)

Private IPv4 addresses
192.168.0.23

Public DNS
ec2-44-202-184-29.compute-1.amazonaws.com | [open address](#)

Elastic IP addresses
-

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.

STEP 8 Request the Peering Connection from Region A

- Switch back to Region A → VPC Console → Peering connections → Create peering connection.
- Name: base-to-regionb-peering
- VPC (Requester): base-vpc
- Account: My account
- Region: Another Region → select Region B
- VPC (Acceptor) ID: enter the VPC ID of peer-vpc-region-b
- Click Create peering connection.

SCREENSHOT: Cross-region peering request created

Create peering connection

A VPC peering connection is a networking connection between two VPCs that enables you to route traffic between them privately. [Info](#)

Peering connection settings

Name - *optional*

Create a tag with a key of 'Name' and a value that you specify.

us-east-2-to-us-east-1

Select a local VPC to peer with

VPC ID (Requester)

vpc-01de7bd764413ad71 (ohio-vpc)

VPC CIDRs for vpc-01de7bd764413ad71 (ohio-vpc)

CIDR	Status	Status reason
10.0.0.0/24	✓ Associated	-

Account

- ☒ My account
☐ Another account

Region

- ☐ This Region (us-east-2)
☒ Another Region

United States (N. Virginia) (us-east-1)

VPC ID (Acceptor)

vpc-074b41b4d7bee7853

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

Value - *optional*

Q us-east-2-to-us-east-1

Remove

Add new tag

You can add 49 more tags.

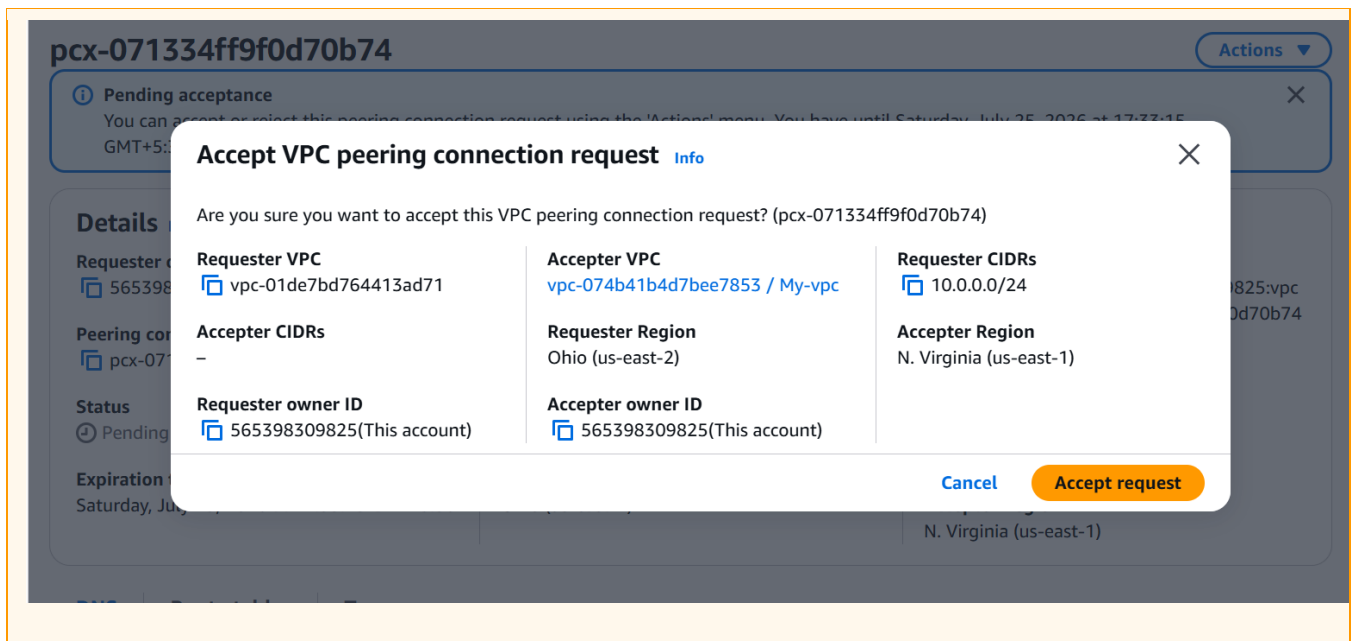
Cancel

Create peering connection

STEP 9 Accept the Peering Connection in Region B

- Switch console to Region B → VPC Console → Peering connections.
- Select the pending request → Actions → Accept request → confirm.

SCREENSHOT: Peering connection accepted



STEP 10 Update Route Tables on Both Sides

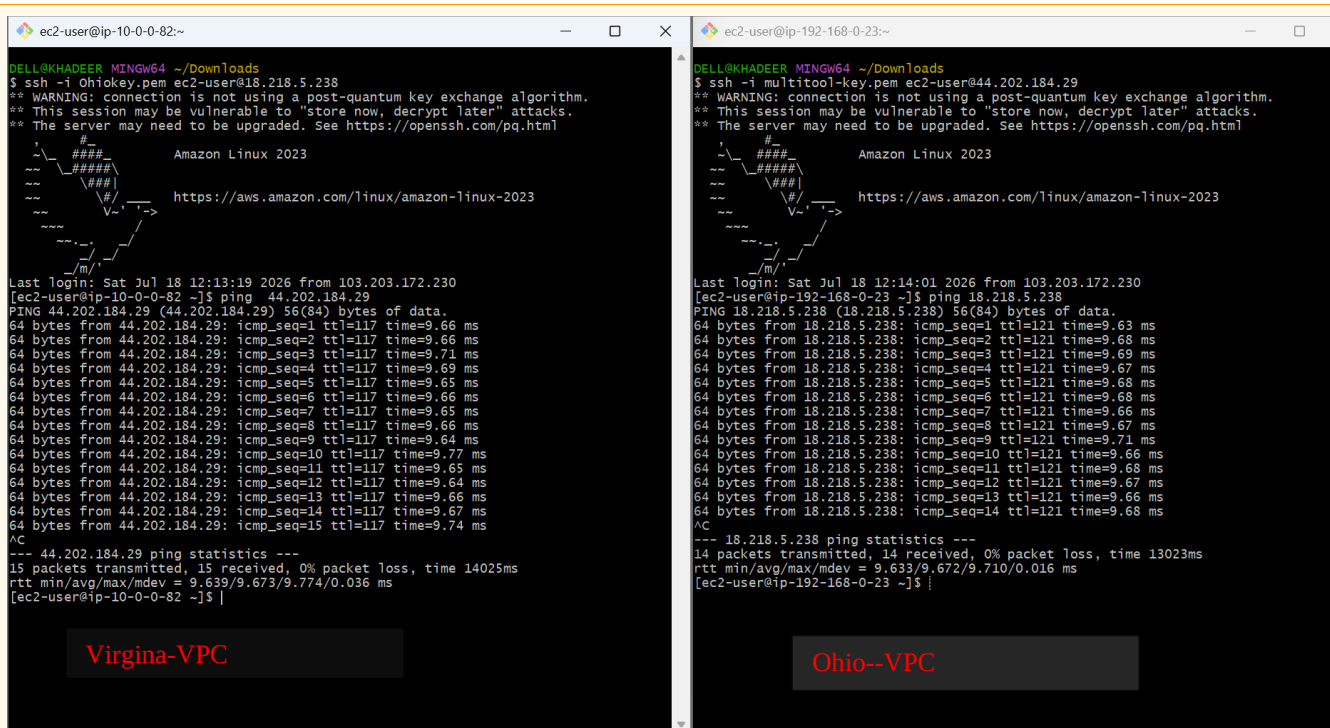
- Region A: base-public-rt and base-private-rt → add route 192.168.0.0/24 → Target: the peering connection.
- Region B: the route table of peer-vpc-region-b → add route 10.0.0.0/24 → Target: the peering connection.

STEP 11 Update Security Groups and Test Connectivity

- Allow inbound ICMP/SSH from the peer CIDR (10.1.0.0/16 or 10.0.0.0/16) on the relevant security groups.
- From an instance in my-Ohiovpc, ping or SSH to an instance's private IP in virgina-vpc-region-b to confirm connectivity.

```
ping <private-ip-in-region-b>
ssh -i key.pem ec2-user@<private-ip-in-region-b>
```

 **SCREENSHOT: Cross-region connectivity verified**



NOTE: Cross-Region peering incurs data transfer charges between Regions; VPC CIDR ranges must not overlap.

4. Enable VPC Peering — Cross-Account

This section peers base-vpc (your account) with a VPC in a friend's/colleague's AWS account (VPC-C). Coordinate the steps below together.

STEP 12 Friend Creates Their VPC and Shares the Account ID

- Friend creates a VPC named peer-vpc-account-c with CIDR 10.2.0.0/16 in their own account.
- Friend shares their 12-digit AWS Account ID and the VPC ID of peer-vpc-account-c with you.
- VPC Console (your account) → Peering connections → Create peering connection.
- Name: base-to-friend-peering
- VPC (Requester): base-vpc
- Account: Another account
- Account ID: enter your friend's Account ID
- VPC (Acceptor) ID: enter peer-vpc-account-c's VPC ID
- Click Create peering connection.

 **SCREENSHOT: Cross-account peering request created.**

VPC > Peering connections > Create peering connection

Create peering connection

A VPC peering connection is a networking connection between two VPCs that enables you to route traffic between them privately. [Info](#)

Peering connection settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

Select a local VPC to peer with

VPC ID (Requester)
vpc-0eef72fbace051cfb (vpc-1)

VPC CIDRs for vpc-0eef72fbace051cfb (vpc-1)

CIDR	Status	Status reason
172.168.0.0/24	Associated	-

Select another VPC to peer with

Account
☐ My account
☒ Another account

Account ID
565398309825

Region
us-east-2

STEP 14 Friend Accepts the Peering Connection

- Friend logs into their own account → VPC Console → Peering connections.
- Friend selects the pending request from your account → Actions → Accept request.

SCREENSHOT: Friend accepts the request

Peering connections (1/3) [Info](#)

Find peering connections by attribute or tag

Name	Peering connection ID	Status	Requester VPC	Accepter VPC
khadeer-mahesh	pcx-053a8e5109425389f	Pending	vpc-0eef72fbace051cfb	vpc-01de7bd764413ad71 / ohio-vpc
khadeer-javed				vpc-01de7bd764413ad71 / ohio-vpc
us-east-2-to-us-east-2				vpc-074b41b4d7be1

Accept VPC peering connection request

Are you sure you want to accept this VPC peering connection request? (pcx-053a8e5109425389f / khadeer-mahesh-peering)

Requester VPC vpc-0eef72fbace051cfb	Accepter VPC vpc-01de7bd764413ad71 / ohio-vpc	Requester CIDRs 172.168.0.0/24
Accepter CIDRs -	Requester Region Stockholm (eu-north-1)	Accepter Region Ohio (us-east-2)
Requester owner ID 385936845252	Accepter owner ID 565398309825(This account)	

[Cancel](#) [Accept request](#)

Details

Requester owner ID 385936845252	Requester VPC vpc-0eef72fbace051cfb
Peering connection ID pcx-053a8e5109425389f	Accepter VPC vpc-01de7bd764413ad71 / ohio-vpc

STEP 15 Update Route Tables on Both Sides

- Your account: base-public-rt / base-private-rt → add route 10.0.0.0/24 → Target: the peering connection.
- Friend's account: their route table → add route 192.168.1.0/24 → Target: the peering connection.

SCREENSHOT: Cross-account routes added

Details [Info](#)

Route table ID

 rtb-06aadb80da92fd54a

VPC

 vpc-01de7bd764413ad71 | ohio-vpc

Main

 No

Owner ID

 565398309825

Explicit subnet associations

 subnet-0e7eecb8624112b26 /
 pubsubnet

Edge associations

–

Routes

Subnet associations

Edge associations

Route propagation

Tags

Related resources

Routes (4)

 Filter routes

Both ▼

[Edit routes](#)

< 1 > 

Destination ▼	Target ▼	Status ▼	Propagated ▼	Route Origin ▼
0.0.0.0/0	igw-07d4a51e63615e5da	✓ Active	No	Create Route
10.0.0.0/24	local	✓ Active	No	Create Route Table
172.168.0.0/24	pcx-053a8e5109425389f	✓ Active	No	Create Route
192.168.1.0/24	pcx-097b7a4788634a0c9	✓ Active	No	Create Route

STEP 16 Update Security Groups and Test Connectivity

- Both sides allow inbound traffic (ICMP/SSH) from the peer's CIDR range.
- Test connectivity between an instance in base-vpc and an instance in the friend's peer-vpc-account-c.

 **SCREENSHOT: Cross-account connectivity verified**

The diagram shows a network of lines and nodes. At the top, there is a horizontal line with a '#' symbol above it. Below this, there are several nodes connected by lines. Some nodes are labeled with '#', and one is labeled 'm'. There are also some symbols that look like 'V' and 'V~'. The network is complex, with many branches and loops. The lines are of different colors, including red, blue, and black. The overall structure is somewhat abstract and difficult to interpret without more context.

NOTE: Cross-account peering never grants access beyond networking; each account still manages its own IAM permissions and resources independently.

5. Set Up a VPC Transit Gateway

A Transit Gateway acts as a central hub so multiple VPCs can reach each other without a full mesh of point-to-point peering connections.

STEP 17 Create the Transit Gateway

- VPC Console → Transit Gateways → Create transit gateway.
- Name: my-Transiet-gw
- Amazon side ASN: leave default (64512) unless a specific ASN is required.
- Default route table association/propagation: leave enabled (default behavior).
- Click Create transit gateway and wait until State = Available.

 SCREENSHOT: Transit Gateway available

tgw-07eab125339a55720 / my-Transiet-gw [Info](#) [Actions](#)

Details

Transit gateway ID tgw-07eab125339a55720	State Available	Amazon ASN 4200000000	DNS support Enable
Transit gateway ARN arn:aws:ec2:us-east-2:565398309825:transit-gateway/tgw-07eab125339a55720	Default association route table Enable	Association route table ID tgw-rtb-04b15491261bf721c	Auto accept shared attachments Disable
Owner ID 565398309825	Default propagation route table Enable	Propagation route table ID tgw-rtb-04b15491261bf721c	VPN ECMP support Enable
Description -	Transit gateway CIDR blocks -	Security Group Referencing support Disable	Metering policy ID -
Multicast support Disable	Encryption support Disabled	Encryption state message -	

[Flow logs](#) [Sharing](#) [Tags](#)

STEP 18 Create Transit Gateway Attachments

- Transit Gateway Attachments → Create transit gateway attachment.
- Attachment 1: Transit Gateway = my-transiet-gwAttacements-vpc1, VPC = my-vpc1,
- Attachment 2: Transit Gateway my-transiet-gwAttacements-vpc2, VPC = my-vpc2 or another VPC you want on the hub.
- Attachment 2: Transit Gateway my-transiet-gwAttacements-vpc3, VPC = my-vpc3

SCREENSHOT: Transit Gateway attachments created

Transit gateway attachments (3) [Info](#) [Actions](#) [Create transit gateway attachment](#)

Find transit gateway attachment by attribute or tag

☐	Name	Transit gateway attachment ID	Transit gateway ID	State	Resource ...
☐	my-transient-gw-Attacements-vpc3	tgw-attach-0430c4c51495e5578	tgw-07eab125339a55720	Available	VPC
☐	my-transient-gw-Attacements-vpc2	tgw-attach-0962f95df2366ba7f	tgw-07eab125339a55720	Available	VPC
☐	my-transient-gw-Attacements-vpc1	tgw-attach-0b22f9f56e1fef6ff	tgw-07eab125339a55720	Available	VPC

Select a transit gateway attachment

Your VPCs

[VPCs](#) [VPC encryption controls](#)

Your VPCs (4) [Info](#) [Actions](#) [Create VPC](#)

Last updated 1 minute ago

Find VPCs by attribute or tag

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Pu
☐	ohio-vpc1	vpc-01de7bd764413ad71	Available	-	-	Off
☐	ohio-vpc3	vpc-02313258e4d327eac	Available	-	-	Off
☐	-	vpc-0b4fcca443abfb569	Available	-	-	Off
☐	ohio-vpc2	vpc-0b0ef358f4aaad904	Available	-	-	Off

STEP 19 Update VPC Route Tables to Point to the Transit Gateway

- base-private-rt → add route for the other VPC's CIDR → Target: base-tgw.
- Repeat on the other attached VPC's route table, pointing back to base-vpc's CIDR → Target: base-tgw.



SCREENSHOT: Routes pointing to Transit Gateway

Route tables (3) [Info](#)

Q Find route tables by attribute or tag

Name : vpc1-RT X

Name : vpc2-RT X

Name : vpc3-RT X

Clear filters

Last updated 1 minute ago

Actions ▼

Create route table

☐	Name ▼	Route table ID ▼	Explicit subnet associ... ▼	Edge associations ▼	Main ▼	VPC
☐	vpc1-RT	rtb-06aadb80da92fd54a	subnet-0e7eecb8624112...	–	No	vpc-01de7bd7
☐	vpc2-RT	rtb-0f4832e727e1b087c	–	–	No	vpc-0b0ef358f
☐	vpc3-RT	rtb-06a696db3c06c92e7	–	–	No	vpc-02313258

STEP 20 Verify the Transit Gateway Route Table

- Transit Gateway Route Tables → select the default association route table → Routes tab.
- Confirm both attachments appear with their associated CIDRs propagated.

STEP 21 Test Connectivity Through the Transit Gateway

- From an instance in one attached VPC, ping/SSH an instance's private IP in the other attached VPC.

 SCREENSHOT: Transit Gateway connectivity verified

Connection From Vpc1 to Vpc2 And Vpc3

```

DELL@KHADEER MINGW64 ~/Downloads
$ ssh -i Ohiokey.pem ec2-user@3.16.23.22
The authenticity of host '3.16.23.22 (3.16.23.22)' can't be established.
ED25519 key fingerprint is: SHA256:m1dx6b7H14Dk6uK5rP5W0uBN595+NTvtPeE9WmtEJQ
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:36: 18.218.5.238
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.16.23.22' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

#_
~\##### Amazon Linux 2023
~~\#####\
~~\###|
~~\#/
~~V~'~>
~~~
~~~.~.~
~~~\m/'

Last login: Sat Jul 18 14:01:38 2026 from 103.203.172.230

```

```
[ec2-user@ip-10-0-0-82 ~]$ ping 18.118.112.166
PING 18.118.112.166 (18.118.112.166) 56(84) bytes of data.
64 bytes from 18.118.112.166: icmp_seq=1 ttl=126 time=0.245 ms
64 bytes from 18.118.112.166: icmp_seq=2 ttl=126 time=0.237 ms
64 bytes from 18.118.112.166: icmp_seq=3 ttl=126 time=0.260 ms
64 bytes from 18.118.112.166: icmp_seq=4 ttl=126 time=0.222 ms
64 bytes from 18.118.112.166: icmp_seq=5 ttl=126 time=0.284 ms
64 bytes from 18.118.112.166: icmp_seq=6 ttl=126 time=0.249 ms
64 bytes from 18.118.112.166: icmp_seq=7 ttl=126 time=0.242 ms
^C
--- 18.118.112.166 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6215ms
rtt min/avg/max/mdev = 0.222/0.248/0.284/0.018 ms
[ec2-user@ip-10-0-0-82 ~]$ ping 3.16.213.163
PING 3.16.213.163 (3.16.213.163) 56(84) bytes of data.
64 bytes from 3.16.213.163: icmp_seq=1 ttl=126 time=0.590 ms
64 bytes from 3.16.213.163: icmp_seq=2 ttl=126 time=0.466 ms
64 bytes from 3.16.213.163: icmp_seq=3 ttl=126 time=0.426 ms
64 bytes from 3.16.213.163: icmp_seq=4 ttl=126 time=0.462 ms
64 bytes from 3.16.213.163: icmp_seq=5 ttl=126 time=0.430 ms
64 bytes from 3.16.213.163: icmp_seq=6 ttl=126 time=0.453 ms
64 bytes from 3.16.213.163: icmp_seq=7 ttl=126 time=0.456 ms
64 bytes from 3.16.213.163: icmp_seq=8 ttl=126 time=0.457 ms
^C
--- 3.16.213.163 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7305ms
rtt min/avg/max/mdev = 0.426/0.467/0.590/0.048 ms
[ec2-user@ip-10-0-0-82 ~]$ |
```

Connection From Vpc2 to Vpc3 And Vpc1

```
DELL@KHADEER MINGW64 ~/Downloads  
$ ssh -i Ohiokey.pem ec2-user@18.118.112.166  
The authenticity of host '18.118.112.166 (18.118.112.166)' can't be established.  
ED25519 key fingerprint is: SHA256:WZHnBkjrbLla5rz1lKi4PqaeBeacgT/+eTb8cc0vJs  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '18.118.112.166' (ED25519) to the list of known hosts.  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html
```

```
#_~\#####Amazon Linux 2023\n~~\\#####\n~~ \\###|\n~~ \\#/\n~~ V~'|'->\n    ~~~~~  
    ._.|_\n      |/_/'_|_  
     /m/'_|_'
```

<https://aws.amazon.com/linux/amazon-linux-2023>


```
[ec2-user@ip-172-168-0-81 ~]$ ping 3.16.23.22
PING 3.16.23.22 (3.16.23.22) 56(84) bytes of data.
64 bytes from 3.16.23.22: icmp_seq=1 ttl=126 time=0.257 ms
64 bytes from 3.16.23.22: icmp_seq=2 ttl=126 time=0.247 ms
64 bytes from 3.16.23.22: icmp_seq=3 ttl=126 time=0.244 ms
64 bytes from 3.16.23.22: icmp_seq=4 ttl=126 time=0.259 ms
64 bytes from 3.16.23.22: icmp_seq=5 ttl=126 time=0.255 ms
64 bytes from 3.16.23.22: icmp_seq=6 ttl=126 time=0.233 ms
64 bytes from 3.16.23.22: icmp_seq=7 ttl=126 time=0.278 ms
64 bytes from 3.16.23.22: icmp_seq=8 ttl=126 time=0.259 ms
^C
--- 3.16.23.22 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7248ms
rtt min/avg/max/mdev = 0.233/0.254/0.278/0.012 ms
[ec2-user@ip-172-168-0-81 ~]$ ping @3.16.213.163
ping: @3.16.213.163: Name or service not known
[ec2-user@ip-172-168-0-81 ~]$ ping 3.16.213.163
PING 3.16.213.163 (3.16.213.163) 56(84) bytes of data.
64 bytes from 3.16.213.163: icmp_seq=1 ttl=126 time=0.450 ms
64 bytes from 3.16.213.163: icmp_seq=2 ttl=126 time=0.440 ms
64 bytes from 3.16.213.163: icmp_seq=3 ttl=126 time=0.414 ms
64 bytes from 3.16.213.163: icmp_seq=4 ttl=126 time=0.434 ms
64 bytes from 3.16.213.163: icmp_seq=5 ttl=126 time=0.475 ms
64 bytes from 3.16.213.163: icmp_seq=6 ttl=126 time=0.437 ms
64 bytes from 3.16.213.163: icmp_seq=7 ttl=126 time=0.451 ms
^C
--- 3.16.213.163 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6239ms
rtt min/avg/max/mdev = 0.414/0.443/0.475/0.017 ms
[ec2-user@ip-172-168-0-81 ~]$
```

Connection From Vpc3 to Vpc2 And Vpc1

[illegible]

6. Set Up a VPC Endpoint

VPC Endpoints let resources in a VPC reach supported AWS services privately, without traversing the internet or a NAT Gateway.

- VPC Console → Endpoints → Create endpoint.
- Service category: AWS services
- Service name: com.amazonaws.<us-east-2>.s3 (Type: Gateway)
- VPC: my-vpc
- Route tables: select private-rt .
- Policy: Full access (or a restricted custom policy).
- Click Create endpoint.

STEP 23 Verify the Gateway Endpoint Route

- Route tables → private-rt → Routes tab → confirm a route to the S3 prefix list via the new endpoint .

STEP 24 Create an Interface Endpoint (e.g., for SSM)

- VPC Console → Endpoints → Create endpoint.
- Service name: com.amazonaws.<region>.ssm (Type: Interface)
- VPC: base-vpc, Subnets: base-private-subnet
- Security group: allow inbound HTTPS (443) from the VPC CIDR.
- Enable Private DNS names: checked.
- Click Create endpoint and wait until State = Available.

STEP 25 Test the VPC Endpoint from a Private Instance

- From the private EC2 instance, run an AWS CLI command that uses the endpoint (e.g., list an S3 bucket) and confirm it succeeds without a NAT Gateway or internet route.

```
aws s3 ls
```

SCREENSHOT: VPC Endpoint connectivity verified —

```
DELL@KHADEER MINGW64 ~/Downloads
$ ssh -i Ohiokey.pem ec2-user@13.59.89.150
The authenticity of host '13.59.89.150 (13.59.89.150)' can't be established.
ED25519 key fingerprint is: SHA256:m1dx6b7H14Dk6uK5rP5WOUBN595+NTvttPeE9WmtEJQ
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:36: 18.218.5.238
  ~/.ssh/known_hosts:40: 3.16.23.22
  ~/.ssh/known_hosts:45: ec2-13-59-89-150.us-east-2.compute.amazonaws.com
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.59.89.150' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

#_
~\#####_      Amazon Linux 2023
~~~\#####\
~~~\###|
~~~\#/
~~~V~'-'>
~~~~
~~~.-.
~~~/_/
~~~/_/m/'

Last login: Mon Jul 20 08:02:46 2026 from 103.203.172.230
https://aws.amazon.com/linux/amazon-linux-2023
```


NOTE: Gateway endpoints (S3, DynamoDB) are free and work via route table entries. Interface endpoints (most other services) create ENIs in your subnets and are billed hourly plus data processed.

8. End-to-End Validation Checklist

#	Check	Expected Result
1	Base VPC subnets	1 public + 1 private subnet created, public subnet auto-assigns public IPs
2	Cross-Region peering	Status = Active; routes present in both Regions; connectivity confirmed
3	Cross-Account peering	Status = Active; routes present in both accounts; connectivity confirmed
4	Transit Gateway	State = Available; attachments Available; routes propagated; connectivity confirmed

#	Check	Expected Result
5	VPC Gateway Endpoint (S3)	State = Available; route table entry present
6	VPC Interface Endpoint	State = Available; private DNS resolves; CLI calls succeed

9. Cleanup (Optional)

- Delete both VPC Endpoints.
- Delete the Transit Gateway attachments, then the Transit Gateway itself.
- Delete both peering connections (cross-region and cross-account).
- Remove the peering/TGW routes added to each route table.
- Terminate any test EC2 instances.
- Delete subnets, route tables, internet gateway, and the VPCs (base-vpc and any peer VPCs you created for testing).

10. Appendix: Quick-Reference CLI Commands

```
# Create VPC and subnets
aws ec2 create-vpc --cidr-block 10.0.0.0/16
aws ec2 create-subnet --vpc-id <vpc-id> --cidr-block 10.0.1.0/24 --availability-
zone <az>

# Cross-region peering request (run in requester region)
aws ec2 create-vpc-peering-connection --vpc-id <requester-vpc-id> \
  --peer-vpc-id <accepter-vpc-id> --peer-region <region-b>

# Cross-account peering request
aws ec2 create-vpc-peering-connection --vpc-id <requester-vpc-id> \
  --peer-vpc-id <accepter-vpc-id> --peer-owner-id <friend-account-id>

# Accept a peering connection (run by the accepter)
aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id <pcx-id>

# Create Transit Gateway and attachment
aws ec2 create-transit-gateway --description base-tgw
aws ec2 create-transit-gateway-vpc-attachment --transit-gateway-id <tgw-id> \
  --vpc-id <vpc-id> --subnet-ids <subnet-id>

# Create a Gateway VPC Endpoint (S3)
aws ec2 create-vpc-endpoint --vpc-id <vpc-id> --service-name
com.amazonaws.<region>.s3 \
  --route-table-ids <rtb-id>

# Create an Interface VPC Endpoint (SSM)
aws ec2 create-vpc-endpoint --vpc-id <vpc-id> --vpc-endpoint-type Interface \
  --service-name com.amazonaws.<region>.ssm --subnet-ids <subnet-id> \
  --security-group-ids <sg-id>
```